6CS012 - Artificial Intelligence and Machine Learning

Worksheet 3: Modeling the Neuron — From MCP to Perceptron

Task 1: MCP Neurons

AND Gate Implementation

```
def MCP_Neurons_AND(X1, X2, T):
    """
    Implements basic AND operations with MCP Neuron for two inputs.
    Arguments:
        X1 (1D array): An array of binary values.
        X2 (1D array): An array of binary values.
        T  (int/float): Threshold value.
    Returns:
        state_neuron (1D list): Neuron output (1 or 0) for each input pair.
    """
    assert len(X1) == len(X2), "X1 and X2 must have the same length."

    # Step 1: Element-wise sum of inputs
    element_sums = [X1[i] + X2[i] for i in range(len(X1))]

    # Step 2: Apply threshold — fire if sum >= T, else 0
    state_neuron = []
    for s in element_sums:
        if s >= T:
            state_neuron.append(1)
        else:
            state_neuron.append(0)

    return state_neuron


# Sample usage
X1 = [0, 0, 1, 1]
X2 = [0, 1, 0, 1]
T  = 2  # AND requires BOTH inputs to be 1, so threshold = 2

result = MCP_Neurons_AND(X1, X2, T)
print(f"Output of AND gate for inputs {X1} and {X2} with threshold {T}: {result}")
```

```
Output of AND gate for inputs [0, 0, 1, 1] and [0, 1, 0, 1] with threshold 2: [0, 0, 0, 1]
```

OR Gate Implementation

```
def MCP_Neurons_OR(X1, X2, T):
    """
    Implements basic OR operations with MCP Neuron for two inputs.
    Arguments:
        X1 (1D array): An array of binary values.
        X2 (1D array): An array of binary values.
        T  (int/float): Threshold value.
    Returns:
        state_neuron (1D list): Neuron output (1 or 0) for each input pair.
    """
    assert len(X1) == len(X2), "X1 and X2 must have the same length."

    # Step 1: Element-wise sum of inputs
    element_sums = [X1[i] + X2[i] for i in range(len(X1))]

    # Step 2: Apply threshold — fire if sum >= T, else 0
    state_neuron = []
    for s in element_sums:
        if s >= T:
            state_neuron.append(1)
        else:
            state_neuron.append(0)

    return state_neuron


# Sample usage
X1 = [0, 0, 1, 1]
X2 = [0, 1, 0, 1]
T  = 1  # OR fires if AT LEAST ONE input is 1, so threshold = 1

result_or = MCP_Neurons_OR(X1, X2, T)
print(f"Output of OR gate for inputs {X1} and {X2} with threshold {T}: {result_or}")
```

```
Output of OR gate for inputs [0, 0, 1, 1] and [0, 1, 0, 1] with threshold 1: [0, 1, 1, 1]
```

Question 1: Limitations of MCP Neurons

1. **Fixed, hand-crafted weights and threshold** - There is no learning mechanism. Weights and thresholds must be set manually by the designer. The model cannot adjust from data.
2. **Binary inputs and outputs only** - MCP neurons only accept 0 or 1 as inputs and produce 0 or 1 as output. Real-world data is continuous.

3. **No inhibitory weight granularity** - Inhibition is all-or-nothing; any inhibitory input forces output to 0 regardless of other inputs. This is biologically crude and computationally limiting.

Question 2: Can MCP Neuron Solve XOR?

No, a single MCP neuron cannot solve XOR directly. because XOR is not linearly separable

```
# Load the dataset
df_0_1 = pd.read_csv("/content/mnist_0_and_1.csv.csv")
```

```
df_0_1.head()
```

	label	pixel_0	pixel_1	pixel_2	pixel_3	pixel_4	pixel_5	pixel_6	pixel_7	pixel_8	...	pixel_774	pixel_775	pixel_776	pixel_777	pixel_778
0	0	0	0	0	0	0	0	0	0	0	...	0.0	0.0	0.0	0.0	0.0
1	1	0	0	0	0	0	0	0	0	0	...	0.0	0.0	0.0	0.0	0.0
2	1	0	0	0	0	0	0	0	0	0	...	0.0	0.0	0.0	0.0	0.0
3	1	0	0	0	0	0	0	0	0	0	...	0.0	0.0	0.0	0.0	0.0
4	1	0	0	0	0	0	0	0	0	0	...	0.0	0.0	0.0	0.0	0.0

5 rows × 785 columns

Task 2: Perceptron for 0 vs 1 Classification (MNIST)

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt


# Extract features and labels
X = df_0_1.drop(columns=["label"]).values  # 784 pixel values per image
y = df_0_1["label"].values                 # Labels: 0 or 1

print("Feature matrix shape:", X.shape)
print("Label vector shape:  ", y.shape)
```

```
Feature matrix shape: (4638, 784)
Label vector shape:   (4638,)
```

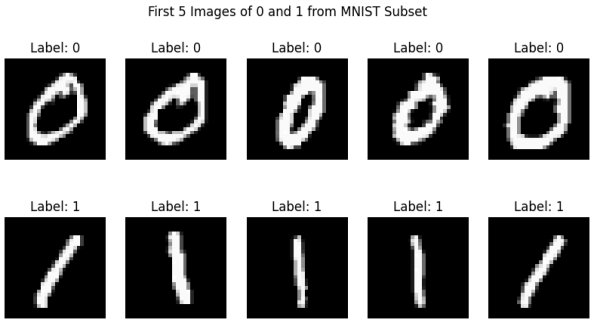
Question 1: What does the shape of X represent?

`X.shape` returns `(n_samples, 784)`. The first dimension is the number of images in the dataset. The second dimension, 784, is the number of features per image — each 28×28 pixel MNIST image is flattened into a 1D array of 784 pixel intensity values.

Question 2: What does the shape of y represent?

`y.shape` returns `(n_samples,)`. It is a 1D array where each element is the true label (0 or 1) corresponding to the image at the same index in X.

```
# Visualize 5 samples of each digit
images_0 = X[y == 0]
images_1 = X[y == 1]

fig, axes = plt.subplots(2, 5, figsize=(10, 5))

if len(images_0) < 5 or len(images_1) < 5:
    print("Error: Not enough images in images_0 or images_1 to plot 5 images.")
else:
    for i in range(5):
        axes[0, i].imshow(images_0[i].reshape(28, 28), cmap="gray")
        axes[0, i].set_title("Label: 0")
        axes[0, i].axis("off")

        axes[1, i].imshow(images_1[i].reshape(28, 28), cmap="gray")
        axes[1, i].set_title("Label: 1")
        axes[1, i].axis("off")

plt.suptitle("First 5 Images of 0 and 1 from MNIST Subset")
plt.show()
```




```
# Initialize weights and bias
weights       = np.zeros(X.shape[1])  # 784 weights, one per pixel
bias          = 0
learning_rate = 0.1
epochs        = 100
```

Question 3: What does the weights array represent?

Each element weights[j] is a scalar that scales the importance of pixel j in making the classification decision. After training, pixels that strongly distinguish 0 from 1 will have large absolute weight values; irrelevant pixels will have weights near zero.

Question 4: Why initialize weights to zero? What effect could this have?

Zero initialization is valid for the perceptron because its weight update rule (w = w + η(y - ŷ)x) is driven purely by prediction errors. There is no symmetry-breaking problem here (unlike deep networks). However, starting at zero means the first prediction for every sample will always be 0 (since dot(X, zeros) + 0 = 0, which maps to class 1 via >= 0 step function — so actually all initial predictions will be class 1). This means the first epoch will produce updates for all class-0 samples, which is expected and fine.

```
def decision_function(X, weights, bias):
    """
    Compute predicted labels for input data using the step activation function.
    Parameters:
        X       : numpy array of shape (n_samples, n_features)
        weights : numpy array of shape (n_features,)
        bias    : scalar
    Returns:
        y_pred_all : numpy array of predicted labels (0 or 1)
    """
    # Compute the net weighted input (linear combination)
    predictions = np.dot(X, weights) + bias

    # Step (Heaviside) activation function: output 1 if >= 0, else 0
    y_pred_all = np.where(predictions >= 0, 1, 0)

    return y_pred_all
```

```
def train_perceptron(X, y, weights, bias, learning_rate=0.1, epochs=100):
    """
    Train the perceptron using the Perceptron Learning Algorithm.
    Parameters:
        X             : numpy array of shape (n_samples, n_features)
        y             : numpy array of shape (n_samples,) — true labels (0 or 1)
        weights       : numpy array of shape (n_features,) — initial weights
        bias          : scalar — initial bias
        learning_rate : float — step size for weight updates
        epochs        : int — maximum number of full passes over the dataset
    Returns:
        weights  : updated weight array after training
        bias     : updated bias after training
        accuracy : fraction of correctly classified samples in the final epoch
    """
    accuracy = 0.0

    for epoch in range(epochs):
        correct = 0
        convergence_flag = True  # Assume convergence until a weight update occurs

        for i in range(len(X)):
            # Step 1: Compute the net weighted input
            output = np.dot(X[i], weights) + bias

            # Step 2: Apply step activation function
            y_hat = 1 if output >= 0 else 0

            # Step 3: Update weights and bias only if prediction is wrong
            if y_hat != y[i]:
                error = y[i] - y_hat          # +1 or -1
                weights += learning_rate * error * X[i]
                bias    += learning_rate * error
                convergence_flag = False       # At least one update occurred
            else:
                correct += 1

        accuracy = correct / len(X)
```



```
        # Early stopping: if no updates occurred this epoch, the model has converged
        if convergence_flag:
            print(f"Converged at epoch {epoch + 1}")
            break

    return weights, bias, accuracy
```

Question 5: What does `output = np.dot(X[i], weights) + bias` do?

It computes the dot product of the input vector `X[i]` (784 pixel values) with the weight vector, then adds the bias. The result is the net weighted sum `z = Σ(wj * xj) + b`. This is the linear combination that represents how strongly the current weights vote for class 1. The activation function then thresholds this value.

Question 6: What happens when the prediction is wrong? How are weights and bias updated?

If `y_hat != y[i]`, the error `(y[i] - y_hat)` is computed. This error is either +1 (predicted 0, true 1) or -1 (predicted 1, true 0). Each weight is then nudged by `η * error * xj`. If a pixel's value is 0, it contributes nothing to the update — only active pixels shift the weights. The bias is shifted by `η * error` independently of input values. The net effect moves the decision boundary toward correctly classifying the misclassified sample.

Question 7: Why is final accuracy important and what is expected?

Final accuracy tells you whether the perceptron has learned a valid decision boundary for the training data. For 0 vs 1 classification, these two digit classes are visually and numerically very distinct (0 is a closed loop, 1 is a vertical stroke), making them nearly linearly separable in pixel space. You should expect accuracy close to or at 100%, often converging before all epochs are exhausted.

```
# Train the perceptron
weights, bias, accuracy = train_perceptron(X, y, weights, bias)
print("The Final Accuracy is: ", accuracy)
```

```
Converged at epoch 8
The Final Accuracy is:  1.0
```

```
# Evaluate predictions on full dataset
predictions = np.dot(X, weights) + bias
y_pred = np.where(predictions >= 0, 1, 0)

final_accuracy = np.mean(y_pred == y)
print(f"Final Accuracy: {final_accuracy:.4f}")

# Visualize misclassified images
misclassified_idx = np.where(y_pred != y)[0]

if len(misclassified_idx) > 0:
    fig, axes = plt.subplots(2, 5, figsize=(10, 5))
    for ax, idx in zip(axes.flat, misclassified_idx[:10]):
        ax.imshow(X[idx].reshape(28, 28), cmap="gray")
        ax.set_title(f"Pred: {y_pred[idx]}, True: {y[idx]}")
        ax.axis("off")
    plt.suptitle("Misclassified Images")
    plt.show()
else:
    print("All images were correctly classified!")
```

```
Final Accuracy: 1.0000
All images were correctly classified!
```

Question 8: What does `misclassified_idx` store and how is it used?

`misclassified_idx` is a numpy array of integer indices where `y_pred[i] != y[i]`. `np.where` scans the boolean array `(y_pred != y)` and returns the positions where the condition is True. These indices are then used to retrieve the corresponding images from `X` (via `X[idx]`) and their predicted vs true labels for visualization.

Question 9: How do you interpret "All images were correctly classified!"?

It means the perceptron found a linear decision boundary (hyperplane in 784-dimensional pixel space) that perfectly separates all class-0 samples from all class-1 samples in the training set. This confirms the data is linearly separable for this pair of digits. Note: this is on training data — it says nothing about generalization to unseen data without a separate test set.

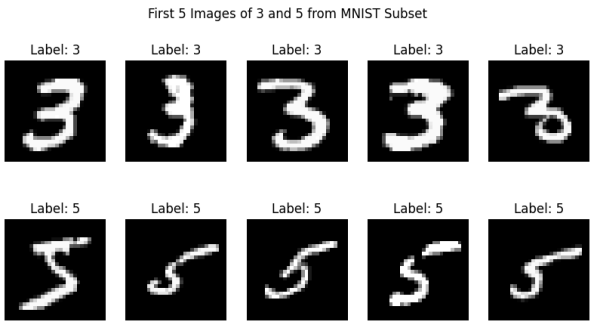
Task 3: Perceptron for 3 vs 5 Classification (MNIST)

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Load the 3 vs 5 dataset
df_3_5 = pd.read_csv("/content/MNIST 3 and 5 Data.csv")

X_35 = df_3_5.drop(columns=["label"]).values
y_35 = df_3_5["label"].values

# Remap labels: treat 3 as 0, 5 as 1 for binary perceptron
y_35_binary = np.where(y_35 == 5, 1, 0)

print("Feature matrix shape:", X_35.shape)
print("Label vector shape:  ", y_35.shape)
print("Unique original labels:", np.unique(y_35))
```

```
Feature matrix shape: (2741, 784)
Label vector shape:   (2741,)
Unique original labels: [3 5]
```

```
# Visualize 5 samples of each digit (3 and 5)
images_3 = X_35[y_35 == 3]
images_5 = X_35[y_35 == 5]
```

```
fig, axes = plt.subplots(2, 5, figsize=(10, 5))

if len(images_3) < 5 or len(images_5) < 5:
    print("Error: Not enough images to plot.")
else:
    for i in range(5):
        axes[0, i].imshow(images_3[i].reshape(28, 28), cmap="gray")
        axes[0, i].set_title("Label: 3")
        axes[0, i].axis("off")

        axes[1, i].imshow(images_5[i].reshape(28, 28), cmap="gray")
        axes[1, i].set_title("Label: 5")
        axes[1, i].axis("off")

plt.suptitle("First 5 Images of 3 and 5 from MNIST Subset")
plt.show()
```



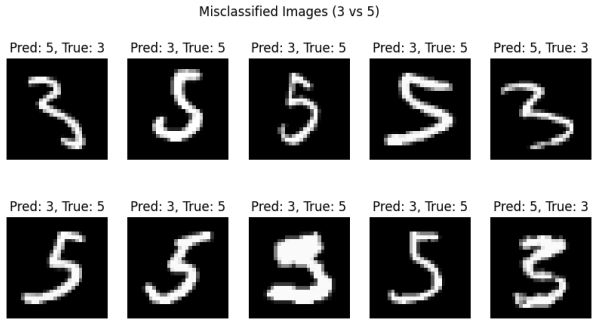

```
# Initialize weights and bias for 3 vs 5
weights_35       = np.zeros(X_35.shape[1])
bias_35          = 0
learning_rate_35 = 0.1
epochs_35        = 100

# Train
weights_35, bias_35, accuracy_35 = train_perceptron(
    X_35, y_35_binary, weights_35, bias_35,
    learning_rate=learning_rate_35, epochs=epochs_35
)
print("The Final Accuracy is: ", accuracy_35)
```

```
The Final Accuracy is:  0.9857716161984678
```

```
# Evaluate predictions
predictions_35 = np.dot(X_35, weights_35) + bias_35
y_pred_35 = np.where(predictions_35 >= 0, 1, 0)

final_accuracy_35 = np.mean(y_pred_35 == y_35_binary)
print(f"Final Accuracy (3 vs 5): {final_accuracy_35:.4f}")

# Visualize misclassified images
misclassified_idx_35 = np.where(y_pred_35 != y_35_binary)[0]

if len(misclassified_idx_35) > 0:
    fig, axes = plt.subplots(2, 5, figsize=(10, 5))
    for ax, idx in zip(axes.flat, misclassified_idx_35[:10]):
        ax.imshow(X_35[idx].reshape(28, 28), cmap="gray")
        # Reverse-map binary label back to original digit for display clarity
        original_pred  = 5 if y_pred_35[idx] == 1 else 3
        original_true  = 5 if y_35_binary[idx] == 1 else 3
        ax.set_title(f"Pred: {original_pred}, True: {original_true}")
        ax.axis("off")
    plt.suptitle("Misclassified Images (3 vs 5)")
    plt.show()
else:
    print("All images were correctly classified!")
```



```
Final Accuracy (3 vs 5): 0.9869
```



Conclusion: 3 vs 5 Classification